

PROGRAMAÇÃO ORIENTADA A OBJETOS 2

Sistema Financeiro

Aplicativo de Conta Bancária Digital

Python • POO • MySQL

Integrantes

Arthur Taíque • Iuri Rodrigues • Matheus Rodrigues
Pedro Henrique • João Dadalto • Hélio Neto

Junho de 2026

Objetivo e Funcionalidades

O que o sistema faz

- **Aplicativo bancário digital em modo texto (terminal), com dados persistidos em MySQL**
- **Funcionalidades do Cliente:**
 - Cadastro com validação de senha (confirmação obrigatória)
 - Login por e-mail e senha (e-mail é o identificador único)
 - Consulta de saldo, extrato completo e transferências (depósito / saque)
- **Funcionalidades do Gerente (perfil administrativo):**
 - Painel exclusivo com CRUD de usuários (listar, atualizar, excluir)
- **Toda movimentação é registrada no histórico com data e hora automáticas**

Arquitetura em Camadas

Separação de responsabilidades

main.py

Camada de Apresentação — menus, telas e controle de sessão



Usuario.py + ContaBancaria.py

Camada de Negócio — classes POO com as regras do sistema



conexao.py + script.sql (MySQL)

Camada de Dados — conexão e estrutura do banco



Cada camada só conversa com a de baixo: o menu não contém SQL nem regra financeira.

Classe Usuario

Cadastro e autenticação — Usuario.py

- **Atributos:** id, nome, email, _senha (protegido) e perfil_id
- **cadastrar():** valida a confirmação da senha em laço
 - Cria o usuário E a conta (saldo R\$ 0,00) juntos — relação 1 : 1
- **login():** consulta parametrizada (%s) evita SQL Injection
 - Se autenticou, o objeto é preenchido com os dados do banco

```
# vínculo 1:1 – a conta nasce junto com o usuário
cursor.execute("INSERT INTO usuarios (nome, email, senha, perfil_id) VALUES (%s,%s,%s,%s)", ...)
usuario_id = cursor.lastrowid # id gerado pelo AUTO_INCREMENT
cursor.execute("INSERT INTO contas (usuario_id, tipo_conta_id, saldo) VALUES (%s, 1, 0.00)", ...)
conn.commit() # efetiva as duas gravações juntas
```

Encapsulamento na Prática

O conceito central de POO do projeto

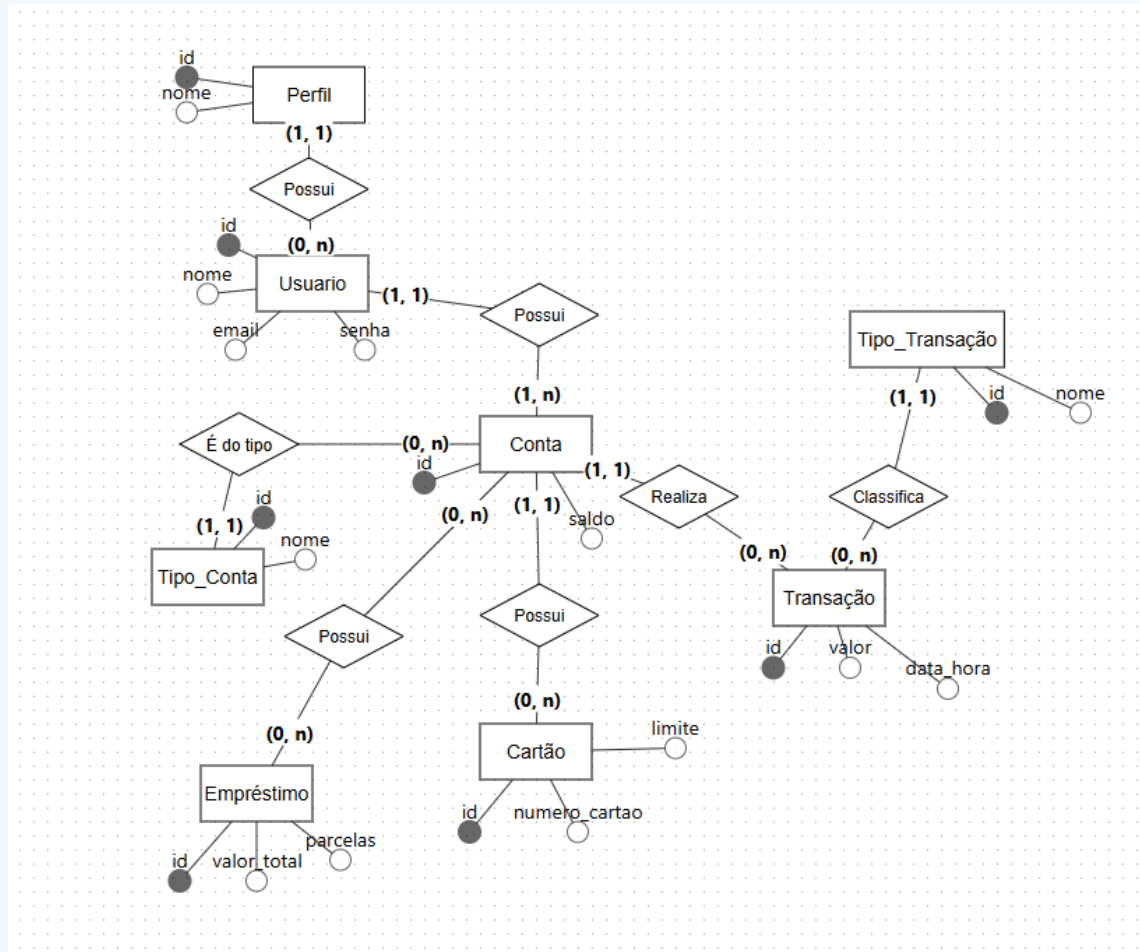
- Atributos protegidos com underscore: `_senha` e `_id_conta`
- O saldo é uma `@property` **SOMENTE LEITURA**:
 - Não existe setter — é impossível fazer `conta.saldo = 1000000`
 - Cada leitura consulta o valor real no banco (nunca fica desatualizado)

```
class ContaBancaria:
    @property
    def saldo(self):
        """Somente leitura: sempre busca o valor real no banco."""
        conn = obter_conexao()
        cursor = conn.cursor()
        cursor.execute("SELECT saldo FROM contas WHERE id = %s", (self._id_conta,))
        resultado = cursor.fetchone()
        return float(resultado[0]) if resultado else 0.0
```

Banco de Dados — Modelo Entidade-Relacionamento

script.sql + MER.png

- USUARIO (1) — (1) CONTA
- CONTA (1) — (N) TRANSACAO
- Tabelas de domínio: perfis,
- tipos_conta, tipos_transacao
- Futuras: cartoes, emprestimos
- email UNIQUE (login único)
- data_hora automática
- (CURRENT_TIMESTAMP)



Principais Regras de Negócio

E onde cada uma foi implementada

✓ **E-mail único identifica o usuário**

» coluna UNIQUE no banco + try/except no cadastro

✓ **Cada usuário tem exatamente 1 conta**

» INSERT duplo no cadastrar() via lastrowid

✓ **Senha e confirmação devem coincidir**

» laço while True na tela de cadastro

✓ **Saldo/Extrato/Transferência exigem login**

» menu adaptativo: opções só existem logado

✓ **Depósitos e saques só com valor positivo**

» validação valor ≤ 0 em realizar_transacao()

✓ **Saque só com saldo suficiente**

» verificação `self.saldo < valor` antes do débito

✓ **Toda transação vai para o extrato**

» registrar_historico() após cada operação

Classe ContaBancaria

Operações financeiras — ContaBancaria.py

- **realizar_transacao(): 4 validações em sequência**
 - 1) sinal + ou - 2) valor numérico 3) valor > 0 4) saldo suficiente (saque)
- **Operação válida = UPDATE do saldo + INSERT no histórico**
- **mostrar_extrato(): INNER JOIN com tipos_transacao, mais recentes primeiro**

```
if sinal == '+':
    novo_saldo = self.saldo + valor          # DEPÓSITO
else:
    if self.saldo < valor:                    # regra de negócio
        print("[ERRO] Saldo insuficiente para realizar o saque.")
        return
    novo_saldo = self.saldo - valor          # SAQUE
cursor.execute("UPDATE contas SET saldo = %s WHERE id = %s", ...)
self.registrar_historico(tipo_id, valor)     # tudo vai para o extrato
```

Fluxo do Sistema e Controle de Sessão

main.py

- **user_logado** guarda o objeto **Usuario** autenticado (**None** = visitante)
- **Menu adaptativo em dois estados:**
 - Deslogado → (1) Cadastro (2) Login (0) Sair
 - Logado → (3) Saldo (4) Extrato (5) Transferência (0) Logout
- **A regra de 'login prévio' vira design: as opções nem aparecem antes do login**
- **Após o login, perfil_id decide o caminho:**
 - perfil_id = 2 (Cliente) → menu financeiro
 - perfil_id = 1 (Gerente) → painel administrativo (CRUD)

Painel do Gerente — CRUD

Perfil administrativo (perfil_id = 1)

C

Create

Cadastro do menu principal (INSERT em usuarios + contas)

R

Read

Listar todos os usuários com id, nome, e-mail e perfil

U

Update

Alterar o nome de um usuário pelo ID

D

Delete

Excluir usuário — a cláusula perfil_id != 1 protege os Gerentes

ON DELETE CASCADE: ao excluir o usuário, o MySQL apaga conta e transações automaticamente.

Demonstração ao Vivo

Roteiro da apresentação prática

- 1. Cadastro de um cliente (mostrando a validação da confirmação de senha)
- 2. Login → o menu muda para as opções financeiras
- 3. Depósito de R\$ 100,00 e saque de R\$ 30,00
- 4. Tentativa de saque de R\$ 500,00 → erro de saldo insuficiente (regra)
- 5. Extrato com data/hora de cada movimentação (INNER JOIN)
- 6. Login do Gerente → listar, atualizar e excluir o cliente de teste
- 7. Exclusão em cascata: conta e transações somem junto com o usuário

Divisão da Apresentação e Próximos Passos

6 integrantes

Arthur Taíque Introdução, objetivo e arquitetura em camadas

Iuri Rodrigues Classe Usuario — cadastro, login e encapsulamento

Matheus Rodrigues Banco de dados, MER e conexao.py

Pedro Henrique Classe ContaBancaria — saldo, transações e extrato

João Dadalto Fluxo do main.py, sessão e painel do Gerente (CRUD)

Hélio Neto Demonstração ao vivo e conclusão

Evoluções futuras

- Hash de senha (bcrypt)
- Cartões e empréstimos
- (tabelas já modeladas)
- Transferência entre contas
- Interface gráfica

Obrigado! Perguntas?